41039 Programming 1

Revision of Week 5: classes, objects, static, enums and switch



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

Outline

1. Classes, objects and static
2. Python classes, enums and switch
3. Write the code
4. Warm-up for today

Fill the blanks: classes and objects

- A class is a **(1)** for objects. Each object made from it is an **(2)** of that class.
- `new` creates the object on the **(3)**, then runs its **(4)**.
- A **(5)** field has one copy for the whole class. An instance field has one copy per object.
- In `this.size = size;` the left side names the **(6)** and the right side names the parameter.

Word bank

a) instance	**f)** field
b) heap	**g)** stack
c) static	**h)** final
d) template	**i)** method
e) constructor	**j)** private

Fill the blanks: classes and objects

- A class is a **(1)** for objects. Each object made from it is an **(2)** of that class.
- `new` creates the object on the **(3)**, then runs its **(4)**.
- A **(5)** field has one copy for the whole class. An instance field has one copy per object.
- In `this.size = size;` the left side names the **(6)** and the right side names the parameter.

Answers

(1) d) template **(2)** a) instance
(3) b) heap **(4)** e) constructor
(5) c) static **(6)** f) field

The variable holds only a **reference**. The object itself lives on the heap.

What does each line hold?

```
class Lamp {
    static int made = 0;
    boolean on;
    String room;
    Lamp(String room) {
        this.room = room;
        made++;
    }
    void toggle() { on = !on; }
}
```

In `main`

```
Lamp a = new Lamp("hall");
Lamp b = new Lamp("desk");
Lamp c = a;
a.toggle();
c.toggle();
b.toggle();
```

Do not guess: draw the objects and the arrows first.

Answer each one (90 seconds, alone)

(a) `a.on`?

(b) `b.on`?

(c) `Lamp.made`?

(d) `c.room`?

(e) `a == c`? `a == b`?

(f) Does `Lamp.toggle();` compile?

What does each line hold?

```
class Lamp {
    static int made = 0;
    boolean on;
    String room;
    Lamp(String room) {
        this.room = room;
        made++;
    }
    void toggle() { on = !on; }
}
```

In `main`

```
Lamp a = new Lamp("hall");
Lamp b = new Lamp("desk");
Lamp c = a;
a.toggle();
c.toggle();
b.toggle();
```

Do not guess: draw the objects and the arrows first.

Answers

(a) `false`: `a` and `c` are one lamp, toggled twice.

(b) `true`: `b` is a different lamp.

(c) 2: two `new`, two constructor runs. `c = a` makes no object.

(d) `hall`.

(e) `true`, `false`: `==` compares references.

(f) No: `toggle` is an instance method. Which lamp would it toggle?

Trace table: one shared total, two objects

```
class Tally {
    static int total = 0;
    int mine = 0;
    void add(int n) { mine += n; total += n; }
}
Tally x = new Tally();
Tally y = new Tally();
for (int i = 1; i <= 4; i++) {
    if (i % 2 == 1) x.add(i);
    else y.add(i);
    if (i == 2) y = x;
}
System.out.println(x.mine + " " + y.mine
                   + " " + Tally.total);
```

Fill the table (2 minutes, alone)

Values at the **end** of each pass. What is printed?

i	call	x.mine	y.mine	total
1	x.add(1)			
2				
3				
4				

Trace table: one shared total, two objects

```
class Tally {
    static int total = 0;
    int mine = 0;
    void add(int n) { mine += n; total += n; }
}
Tally x = new Tally();
Tally y = new Tally();
for (int i = 1; i <= 4; i++) {
    if (i % 2 == 1) x.add(i);
    else y.add(i);
    if (i == 2) y = x;
}
System.out.println(x.mine + " " + y.mine
                   + " " + Tally.total);
```

Fill the table (2 minutes, alone)

Values at the **end** of each pass. What is printed?

i	call	x.mine	y.mine	total
1	x.add(1)	1	0	1
2	y.add(2)	1	**1**	3
3	x.add(3)	4	4	6
4	y.add(4)	8	8	10

Answer

Output: `8 8 10`. After `y = x`, both names reach **one** object, so `y.mine` shows 1. The lost object's 2 stays in `total`: static data belongs to the class.

Predict the output: the pizza that forgot its size

```
class Pizza {
    String size;
    int slices;
    Pizza(String size, int slices) {
        size = size;
        this.slices = slices;
    }
    String describe() {
        return size + " pizza, "
               + slices + " slices";
    }
}
// in main
Pizza p = new Pizza("large", 8);
System.out.println(p.describe());
```

Predict (1 minute, alone)

1. What is printed?
2. Change **one** thing to print `large pizza, 8 slices`.

Predict the output: the pizza that forgot its size

```
class Pizza {
    String size;
    int slices;
    Pizza(String size, int slices) {
        size = size;
        this.slices = slices;
    }
    String describe() {
        return size + " pizza, "
               + slices + " slices";
    }
}
// in main
Pizza p = new Pizza("large", 8);
System.out.println(p.describe());
```

Predict (1 minute, alone)

1. What is printed?

2. Change **one** thing to print `large pizza, 8 slices`.

Answers

1. `null pizza, 8 slices`.

Here `size` means the **parameter**, which hides the field. It is copied onto itself; the field keeps its default, `null`. It compiles.

2. `this.size = size;` as on the next line.

Which of these methods compile?

```
public class Shop {
    static int opened = 0;
    String name;

    void rename(String n) {          // (a)
        name = n;
        opened++;
    }
    static String label() {          // (b)
        return name;
    }
    static String sample() {         // (c)
        Shop s = new Shop();
        return s.name;
    }
}
```

Check (1 minute, alone)

Which methods compile? For each one that does not, what does `javac` say?

Which of these methods compile?

```
public class Shop {
    static int opened = 0;
    String name;

    void rename(String n) {          // (a)
        name = n;
        opened++;
    }
    static String label() {          // (b)
        return name;
    }
    static String sample() {         // (c)
        Shop s = new Shop();
        return s.name;
    }
}
```

Check (1 minute, alone)

Which methods compile? For each one that does not, what does `javac` say?

Answers

(a) Compiles. It runs for one shop: it sees that shop's `name` **and** the shared `opened`.

(b) No: *non-static variable name cannot be referenced from a static context.* No `this`, so no shop.

(c) Compiles and returns `null`: it reads `name` **through a reference** to an object.

Outline

1. Classes, objects and static
2. Python classes, enums and switch
3. Write the code
4. Warm-up for today

Predict the output: three kinds of method

```
class Cup:
    size = "M"
    count = 0
    def __init__(self, drink):
        self.drink = drink
        Cup.count += 1
    def label(self):
        return self.size + " " + self.drink
    @classmethod
    def made(cls): return cls.count
    @staticmethod
    def price(size):
        return {"S": 3, "M": 4, "L": 5}[size]
a = Cup("tea")
b = Cup("coffee")
b.size = "L"
print(a.label(), b.label())
print(Cup.made(), Cup.size)
print(Cup.price(b.size))
```

Predict (90 seconds, alone)

1. What is printed?
2. Which method gets `self`, which `cls`, which neither?

Predict the output: three kinds of method

```python
class Cup:
    size = "M"
    count = 0
    def __init__(self, drink):
        self.drink = drink
        Cup.count += 1
    def label(self):
        return self.size + " " + self.drink
    @classmethod
    def made(cls): return cls.count
    @staticmethod
    def price(size):
        return {"S": 3, "M": 4, "L": 5}[size]
a = Cup("tea")
b = Cup("coffee")
b.size = "L"
print(a.label(), b.label())
print(Cup.made(), Cup.size)
print(Cup.price(b.size))
```

Predict (90 seconds, alone)

1. What is printed?

2. Which method gets `self`, which `cls`, which neither?

Answers

1. `M tea L coffee`, then `2 M`, then `5`.

`b.size = "L"` creates a new attribute **on `b` only**. `a` still finds the class's `"M"`.

2. `label` gets `self` (the object), `made` gets `cls` (the class), `price` gets neither: a static method.

Predict the output: an enum switch that falls through

```
enum Size { SMALL, MEDIUM, LARGE }

static int price(Size s) {
    int p = 0;
    switch (s) {
        case SMALL:  p = 3;
        case MEDIUM: p = 4; break;
        case LARGE:  p = 5;
    }
    return p;
}
// in main
for (Size s : Size.values())
    System.out.print(s + "=" + price(s) + " ");
System.out.println(
    Size.valueOf("LARGE").ordinal());
```

Predict (90 seconds, alone)

1. What is printed?

2. Rewrite `price` with a switch **expression** (Java 14+). What happens if you leave out `LARGE`?

Predict the output: an enum switch that falls through

```
enum Size { SMALL, MEDIUM, LARGE }

static int price(Size s) {
    int p = 0;
    switch (s) {
        case SMALL:  p = 3;
        case MEDIUM: p = 4; break;
        case LARGE:  p = 5;
    }
    return p;
}
// in main
for (Size s : Size.values())
    System.out.print(s + "=" + price(s) + " ");
System.out.println(
    Size.valueOf("LARGE").ordinal());
```

Answers

1. `SMALL=4 MEDIUM=4 LARGE=5 2`. No `break` after `SMALL`, so it falls into `MEDIUM`. `ordinal()` counts from 0.

2. Arrows never fall through, so `SMALL` gives 3:

```
return switch (s) {
    case SMALL  -> 3;
    case MEDIUM -> 4;
    case LARGE  -> 5;
};
```

Leave out `LARGE`: **compile error**, *the switch expression does not cover all possible input values.*

Spot the errors: the player score

✗ Task: each player has a name, points and a history. Expected: `An: 7`, then `[]`.

```
1  class Player:
2      history = []
3      def __init__(name, points):
4          self.name = name
5          self.points = points
6      def add(self, n):
7          points += n
8          self.history.append(n)
9      def __str__(self):
10         print(self.name + ": " + str(self.points))
11 an, binh = Player("An", 0), Player("Binh", 5)
12 an.add(3)
13 an.add(4)
14 print(an)
15 print(binh.history)
```

Find **4** problems. Python stops at the **first** one it meets.

Spot the errors: the player score

✗ Task: each player has a name, points and a history. Expected: `An: 7`, then `[]`.

```
1  class Player:
2      history = []
3      def __init__(name, points):
4          self.name = name
5          self.points = points
6      def add(self, n):
7          points += n
8          self.history.append(n)
9      def __str__(self):
10         print(self.name + ": " + str(self.points))
11 an, binh = Player("An", 0), Player("Binh", 5)
12 an.add(3)
13 an.add(4)
14 print(an)
15 print(binh.history)
```

Find **4** problems. Python stops at the **first** one it meets.

✓ Fixes, in the order Python meets them

- Line 3: no `self`, so *TypeError: ___init___() takes 2 positional arguments but 3 were given*.
- Line 7: `points` is a new local: *UnboundLocalError*. Use `self.points`.
- Line 10: `__str__` must **return** the text. It prints `An: 7`, then a *TypeError*.
- Line 2: one list for **all** players. No error, but it prints `[3, 4]`. Use `self.history = []`.

Pair activity: break the player

Pairs ▪ 3 minutes

The class (all 4 bugs fixed)

```
class Player:
    def __init__(self, name, points):
        self.name = name
        self.points = points
        self.history = []
    def add(self, n):
        self.points += n
        self.history.append(n)
    def __str__(self):
        return self.name + ": " + str(self.points)
```

Your task

1. One call that leaves a player in a state no game should allow.
2. What does `Player("An", 0) == Player("An", 0)` print? Why?
3. Can other code give a player 1000 points without calling `add`? Could it in Java?

Pair activity: break the player

Pairs ▪ 3 minutes

The class (all 4 bugs fixed)

```
class Player:
    def __init__(self, name, points):
        self.name = name
        self.points = points
        self.history = []
    def add(self, n):
        self.points += n
        self.history.append(n)
    def __str__(self):
        return self.name + ": " + str(self.points)
```

One good answer

1. `p.add(-50)` prints `An: -50`: nothing checks `n`. `Player("", 0)` is another.

2. `False`. Without `__eq__`, `==` asks "the **same** object?", like `is`. These are two objects.

3. Yes: `p.points = 1000` works. Python attributes are public. In Java, a `private` field makes the compiler refuse it. That is today's topic.

Which version would you rather debug?

✗ Version 1

```
static final int HEARTS = 0, SPADES = 1,
                 DIAMONDS = 2, CLUBS = 3;
static String colour(int suit) {
    if (suit == HEARTS || suit == DIAMONDS)
        return "red";
    return "black";
}
System.out.println(colour(DIAMONDS));
System.out.println(colour(7));
```

✓ Version 2

```
enum Suit {
    HEARTS, SPADES, DIAMONDS, CLUBS;
    String colour() {
        if (this == HEARTS
                || this == DIAMONDS)
            return "red";
        return "black";
    }
}
System.out.println(Suit.DIAMONDS.colour());
```

Which version would you rather debug?

✗ Version 1

```
static final int HEARTS = 0, SPADES = 1,
                 DIAMONDS = 2, CLUBS = 3;
static String colour(int suit) {
    if (suit == HEARTS || suit == DIAMONDS)
        return "red";
    return "black";
}
System.out.println(colour(DIAMONDS));
System.out.println(colour(7));
```

✓ Version 2

```
enum Suit {
    HEARTS, SPADES, DIAMONDS, CLUBS;
    String colour() {
        if (this == HEARTS
                || this == DIAMONDS)
            return "red";
        return "black";
    }
}
System.out.println(Suit.DIAMONDS.colour());
```

Why version 2

Version 1 prints `red`, then `black` for suit 7, which does not exist, and nothing complains. With the enum, `colour(7)` does not compile (*int cannot be converted to Suit*). Only four suits can exist, `==` is safe, they print as `DIAMONDS` rather than 2, and the method lives with the data.

Outline

1. Classes, objects and static
2. Python classes, enums and switch
3. Write the code
4. Warm-up for today

Write the code: a rectangle that counts itself

Java ▪ alone ▪ 4 minutes. Finish `Rectangle`: set the fields with `this`, count every rectangle made, and compute the area.

Skeleton

```
class Rectangle {
    static int count = 0;
    private int width;
    private int height;
    Rectangle(int width, int height) {
        // TODO: set the fields with this
        // TODO: count this rectangle
    }
    int area() {
        return 0;  // TODO
    }
}
```

Test

```
Rectangle a = new Rectangle(3, 4);
Rectangle b = new Rectangle(5, 5);
System.out.println(a.area() + " " + b.area());
System.out.println(Rectangle.count);
```

Must print

```
12 25
2
```

Write the code: a rectangle that counts itself

Java ▪ alone ▪ 4 minutes. Finish `Rectangle`: set the fields with `this`, count every rectangle made, and compute the area.

Skeleton

```
class Rectangle {
    static int count = 0;
    private int width;
    private int height;
    Rectangle(int width, int height) {
        // TODO: set the fields with this
        // TODO: count this rectangle
    }
    int area() {
        return 0;  // TODO
    }
}
```

✓ One answer

```
class Rectangle {
    static int count = 0;
    private int width;
    private int height;
    Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
        count++;
    }
    int area() {
        return width * height;
    }
}
```

`count++` runs once per `new`. Without `this.`, both areas would be 0.

Write the code: a temperature in Python

Python ▪ alone ▪ 4 minutes. Finish `Temperature`: keep the Celsius value, build one from Fahrenheit with $C = (F - 32) \times 5/9$, and print it to 1 decimal place.

Skeleton

```
class Temperature:
    def __init__(self, celsius):
        pass  # TODO: keep celsius
    @classmethod
    def from_fahrenheit(cls, f):
        pass  # TODO: a new Temperature
    def __str__(self):
        return "?"  # TODO: like "21.5 C"
```

Test

```
print(Temperature(21.5))
print(Temperature.from_fahrenheit(212))
print(Temperature.from_fahrenheit(70))
```

Must print

```
21.5 C
100.0 C
21.1 C
```

Write the code: a temperature in Python

Python ▪ alone ▪ 4 minutes. Finish `Temperature`: keep the Celsius value, build one from Fahrenheit with $C = (F - 32) \times 5/9$, and print it to 1 decimal place.

Skeleton

```
class Temperature:
    def __init__(self, celsius):
        pass  # TODO: keep celsius
    @classmethod
    def from_fahrenheit(cls, f):
        pass  # TODO: a new Temperature
    def __str__(self):
        return "?"  # TODO: like "21.5 C"
```

✓ One answer

```
class Temperature:
    def __init__(self, celsius):
        self.celsius = celsius
    @classmethod
    def from_fahrenheit(cls, f):
        return cls((f - 32) * 5 / 9)
    def __str__(self):
        c = round(self.celsius, 1)
        return str(c) + " C"
```

`cls(...)` calls the constructor. `__str__` **returns** the text.

Write the code: an enum with data

Java ▪ alone ▪ 4 minutes. Finish `Size`: `SMALL`, `MEDIUM` and `LARGE` hold 250, 350 and 500 ml. `bigger()` gives the next size up; `LARGE` stays `LARGE`.

Skeleton

```
enum Size {
    // TODO: give them 250, 350, 500 ml
    SMALL, MEDIUM, LARGE;
    // TODO: a private final field ml,
    // and a constructor that sets it
    int ml() {
        return 0;  // TODO
    }
    Size bigger() {
        return this;  // TODO
    }
}
```

Test

```
System.out.println(Size.SMALL.ml());
System.out.println(Size.SMALL.bigger());
System.out.println(Size.LARGE.bigger().ml());
```

Must print

```
250
MEDIUM
500
```

Write the code: an enum with data

Java ▪ alone ▪ 4 minutes. Finish `Size`: `SMALL`, `MEDIUM` and `LARGE` hold 250, 350 and 500 ml. `bigger()` gives the next size up; `LARGE` stays `LARGE`.

Skeleton

```
enum Size {
    // TODO: give them 250, 350, 500 ml
    SMALL, MEDIUM, LARGE;
    // TODO: a private final field ml,
    // and a constructor that sets it
    int ml() {
        return 0;  // TODO
    }
    Size bigger() {
        return this;  // TODO
    }
}
```

✓ One answer

```
enum Size {
    SMALL(250), MEDIUM(350), LARGE(500);
    private final int ml;
    Size(int ml) { this.ml = ml; }
    int ml() { return ml; }
    Size bigger() {
        if (this == LARGE) return LARGE;
        return values()[ordinal() + 1];
    }
}
```

Values first, then a **semicolon**. Each value calls the constructor once.

Outline

1. Classes, objects and static
2. Python classes, enums and switch
3. Write the code
4. Warm-up for today

Bridge: when anyone can change the count

Last week: a field anyone can write

```
class Stock {
    int count;
}
// anywhere in the program
Stock s = new Stock();
s.count = 5;
s.count -= 8;
System.out.println(s.count);   // -3
```

Think (1 minute)

The rule is: **stock is never negative**. In a 2,000-line shop program, where must you check it?

Bridge: when anyone can change the count

Last week: a field anyone can write

```
class Stock {
    int count;
}
// anywhere in the program
Stock s = new Stock();
s.count = 5;
s.count -= 8;
System.out.println(s.count);   // -3
```

Think (1 minute)

The rule is: **stock is never negative**. In a 2,000-line shop program, where must you check it?

Everywhere `s.count` is written, and you cannot even find them all. Instead, let the class keep its own rule:

```
class Stock {
    private int count;
    boolean take(int n) {
        if (n <= 0 || n > count) return false;
        count -= n;
        return true;
    }
}
```

One place to check, and `s.count -= 8;` no longer compiles outside the class. Today: **contracts**, **valid state**, and classes you can rely on.

You should now be able to...

Classes, objects and static

- ✓ write a class with fields, a constructor and methods
- ✓ use `this` to tell a field from a parameter
- ✓ draw **references** and **objects** separately
- ✓ tell **static** (one per class) from **instance** (one per object)
- ✓ say what a static method cannot see, and why

Python classes, enums and switch

- ✓ use `self`, `cls`, `@classmethod` and `@staticmethod`
- ✓ return a string from `__str__`
- ✓ give an enum fields, a constructor and methods
- ✓ spot fall-through, and use a switch expression (Java 14+)

Next: Week 6 – Building reliable classes

Contracts and valid state, references and object lifetime, shared versus instance data, and composition.